

Code Walkthrough: Weather Advisor Script

Line-by-line notes on `index.py` — for someone new to `dotenv`, APIs, and the Gemini client

1. Imports

```
import requests as req

from google import genai
from dotenv import load_dotenv
import os
```

requests (imported as **req**) is a library used to make HTTP calls to web APIs — here it fetches weather data from OpenWeatherMap. **genai** is Google's SDK for talking to Gemini models. **dotenv** is a small library that reads a **.env** file and loads its contents as environment variables. **os** is Python's built-in module for interacting with the operating system, including reading those environment variables via **os.getenv()**.

2. What is dotenv, and why is it used here?

```
load_dotenv()
```

API keys (like your Weather API key or Gemini API key) are secrets — you never want to hardcode them directly into your script, especially if the code will be shared, uploaded to GitHub, or version-controlled. Instead, you create a file named **.env** in your project folder that looks like this:

```
WEATHER_API_KEY=your_key_here
GEMINI_API_KEY=your_key_here
```

load_dotenv() reads that **.env** file and copies its key-value pairs into your program's environment variables, as if you had set them in your terminal with `export KEY=value`. Once loaded, any part of your script can retrieve them using **os.getenv('KEY_NAME')**. This keeps secrets out of your source code — you'd add **.env** to your **.gitignore** so it's never committed to a repository.

Note: If you forget to call `load_dotenv()`, `os.getenv()` will return `None` for these keys, and your API calls will fail with an authentication error.

3. Taking user input

```
city = input("enter city: ")
```

A simple built-in Python function. It pauses the script, shows the prompt text in the terminal, and waits for the user to type something and press Enter. Whatever they type is returned as a string and stored in **city**.

4. Building the weather API request URL

```
url_weather = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={os.getenv('WEATHER_API_KEY')}"
res_weather = req.get(url_weather)
```

This builds a URL to OpenWeatherMap's 'current weather' endpoint using an f-string (a Python string that lets you insert variables directly inside `{}` braces). The URL has three query parameters: **q** is the city name,

appid is your API key (pulled from the environment via `os.getenv`, exactly as explained above), and **units=metric** tells the API to return temperature in Celsius instead of the default Kelvin. **req.get(url_weather)** then sends an HTTP GET request to that URL and stores the server's response (status code, headers, and body) in **res_weather**.

5. Parsing the response as JSON

```
weather_data = res_weather.json()
```

APIs like this return data as JSON (JavaScript Object Notation) — essentially a structured text format that looks like nested Python dictionaries and lists. **.json()** is a method from the requests library that parses that raw text response into an actual Python dictionary you can index into, e.g.

```
weather_data["main"]["temp"].
```

6. Creating the Gemini client

```
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
```

This creates an object that represents your connection to Google's Gemini API. It authenticates using your Gemini key (again read from the `.env` file). Every subsequent call to Gemini goes through this **client** object.

7. Extracting specific fields from the weather data

```
temp = weather_data["main"]["temp"]
description = weather_data["weather"][0]["description"]
```

OpenWeatherMap's JSON response nests the temperature inside a **"main"** object, and the text description (e.g. "clear sky", "light rain") inside a **"weather"** list — a list because a location can technically have more than one weather condition tag, so **[0]** grabs the first one.

8. Building the prompt for Gemini

```
prompt = f"""You are a Weather Data Analyst.
The current weather in {city} is {temp}°C with {description}.
Based on this data, give me:
1. Outfit suggestions (fabric, layers, etc.)
2. Hydration or dietary tips
3. Any precautions I should take today
"""
```

A multi-line f-string (triple quotes allow the string to span several lines) that inserts the city, temperature, and description into a natural-language instruction. This is the actual prompt that gets sent to the AI model — essentially you're asking it to act as a persona ("Weather Data Analyst") and produce a structured answer.

9. Calling the Gemini model

```
interaction = client.interactions.create(
    model="gemini-3.5-flash",
    input=prompt
)
```

```
print(interaction.output_text)
```

This sends the prompt to the specified Gemini model and gets back a response object, stored in **interaction**. **interaction.output_text** extracts just the generated text reply, which is then printed to the terminal.

Note: The exact method name (`client.interactions.create`) and model string depend on the version of the `google-genai` SDK you have installed — if you get an `AttributeError` here, it usually means the SDK's API surface has changed and you should check the installed package's documentation for the current method (e.g. it may instead be `client.models.generate_content(...)` in some versions).

Summary: The Overall Flow

1. Load secret API keys from a hidden `.env` file → 2. Ask the user for a city → 3. Call the weather API to get live data → 4. Pull out temperature and description → 5. Stuff that data into a prompt → 6. Send the prompt to Gemini → 7. Print the AI-generated advice.